

第十章 内部排序

10.1 概述

10.2 插入排序

10.3 交换排序

10.4 选择排序

10.5 归并排序

10.6 基数排序

10.7 各种内部排序方法比较

本章学习要点 习题与上机作业

10.1 概述

10.1.1 什么是排序

其目的是将一组“**无序**”的记录(数据元素)序列调整为“**有序**”的记录(数据元素)序列，是根据记录(数据元素)关键字的值的**非递减或者非递增**（**递增或者递减**）的关系将记录(数据元素)的次序重新排列。

[例] 将下列关键字记录序列：

52, 49, 80, 36, 14, 58, 61, 23, 97, 75

调整为：

14, 23, 36, 49, 52, 58, 61, 75, 80, 97

10.1.2 排序的分类

[根据排序时文件记录的存放位置]

内部排序：排序过程中将全部记录放在内存中处理。

外部排序：排序过程中需在内外存之间交换信息。

[根据排序前后相同关键字记录的相对次序]

稳定排序：设文件中任意两个记录的关键字值相同，即 $K_i=K_j (i \neq j)$ ，若排序之前记录 R_i 领先于记录 R_j ，排序后这种关系不变(对所有输入实例而言)。

不稳定排序：只要有一个实例使排序算法不满足稳定性要求。

key ptr

[根据存储结构划分排序的种类]

顺序存储

链式存储

地址存储: 待排记录顺序存储, 排序时只对辅助表(关键字+指针)的表目进行物理重排。

[根据内部排序的方法]

插入排序

交换排序

选择排序

归并排序

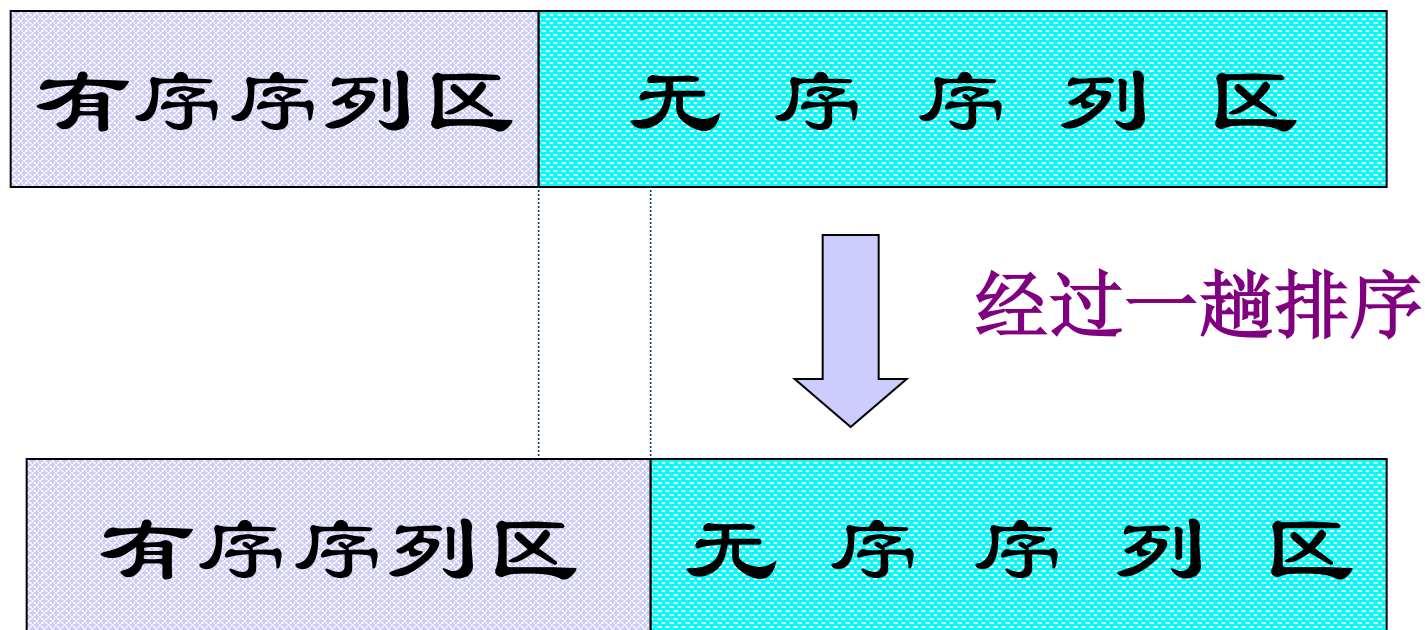
基数排序

[根据排序算法所需的辅助空间]

就地排序: $O(1)$ 非就地排序: $O(n)$ 或与 n 有关



内部排序的过程是一个逐步扩大记录的有序序列长度的过程。



内部排序方法基于不同的“扩大”有序序列长度的方法。



10.1.3 评价排序算法的主要衡量指标(标准)

[时间开销]

考察算法的两个基本操作的次数：

- 比较关键字
- 移动记录

算法时间还与输入实例的初始状态有关时，分情况：

- 最好
- 最坏
- 平均

[空间开销]

所需的辅助空间

[稳定性]

A和B的关键字相等，排序后A、B的先后次序保持不变，则称这种排序算法是稳定的。

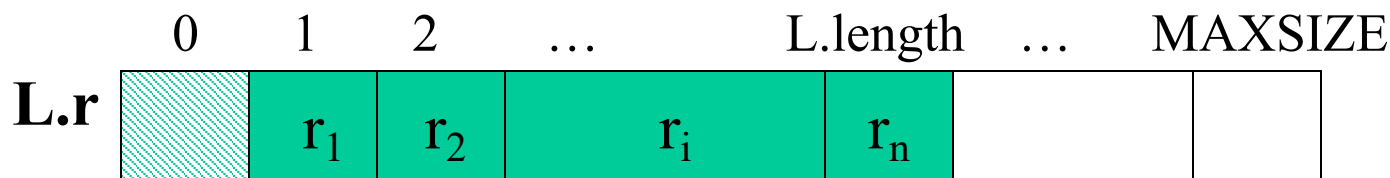
讨论约定

(1)顺序存储

(2)按记录关键字非递减，关键字为整数

```
#define MAXSIZE 20
typedef int KeyType;
typedef struct {
    KeyType key;
    InfoType otherinfo;
} RedType;
typedef struct{
    RedType r[MAXSIZE+1]; //r[0]闲置或作哨兵
    int length;
}SqList;
```

SqList L;



排序算法分类

规则不同

插入排序（直接插入排序，折半插入排序，希尔排序）

交换排序（冒泡排序，快速排序）

选择排序（简单选择排序，树形选择排序，堆排序）

归并排序（2路归并排序）

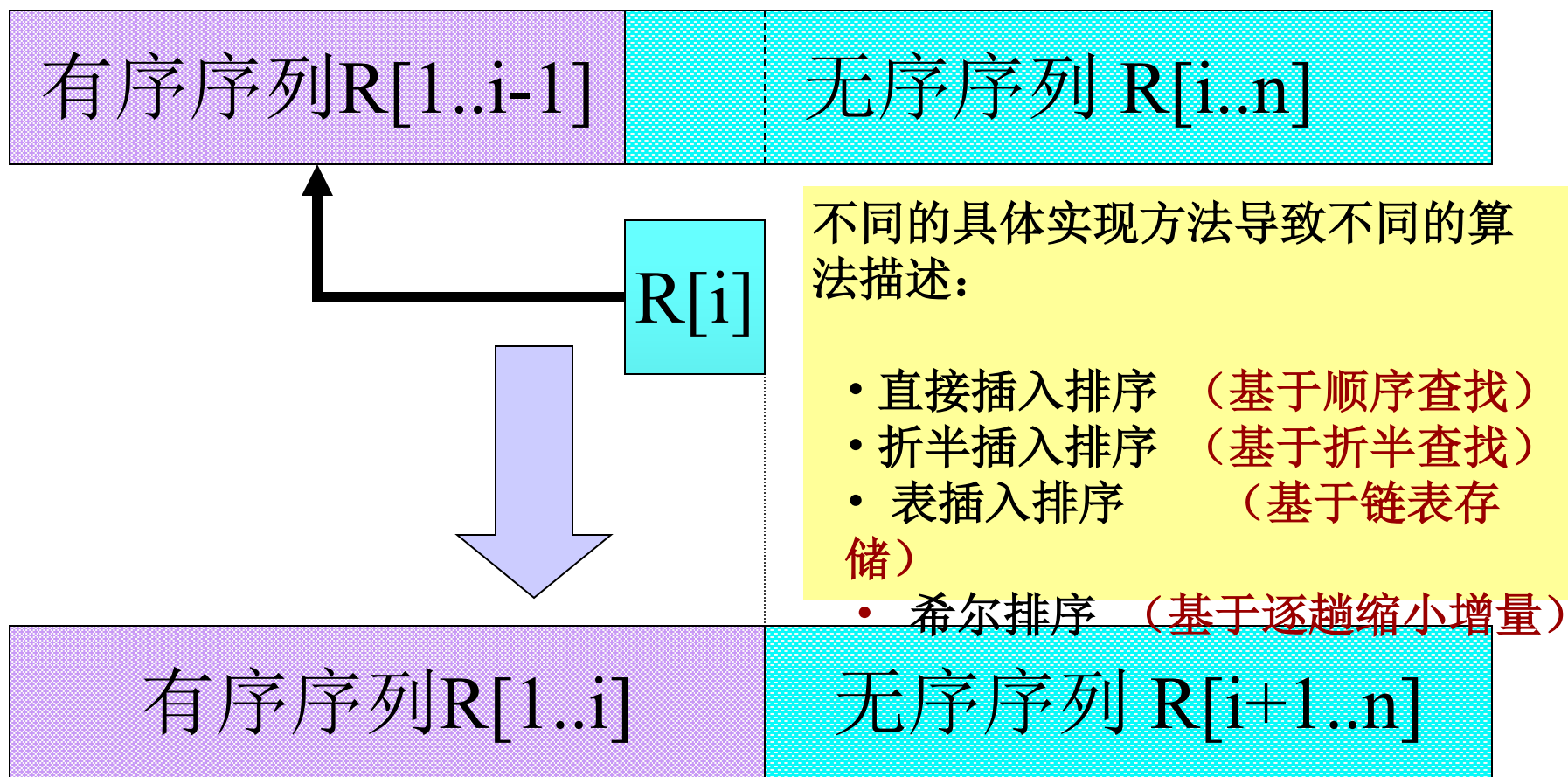
时间复杂度不同

简单排序 $O(n^2)$ （直接插入排序、折半插入排序、冒泡排序、简单选择排序）

先进排序 $O(n\log_2 n)$ （快速排序、树形选择排序、堆排序、2路归并排序）

10.2 插入排序

一趟直接插入排序的基本思想：



10.2.1 直接插入排序（增量法）

利用“顺序查找”实现“在 $R[1..i-1]$ 中查找 $R[i]$ 的插入位置”

[示例] { $R(0)$ $R(-4)$ $R(8)$ $R(1)$ $R(-4)$ $R(-6)$ } $n=6$

$i=1$ [0] -4 8 1 -4 -6

$i=2$ [-4 0] 8 1 -4 -6

$i=3$ [-4 0 8] 1 -4 -6

$i=4$ [-4 0 1 8] -4 -6

$i=5$ [-4 -4 0 1 8] -6

$i=6$ [-6 -4 -4 0 1 8]

稳定排序

[算法思想] 每次使有序区增加一个记录

插入排序的基本步骤:

≤的比较方式，决定了
插入排序是稳定排序

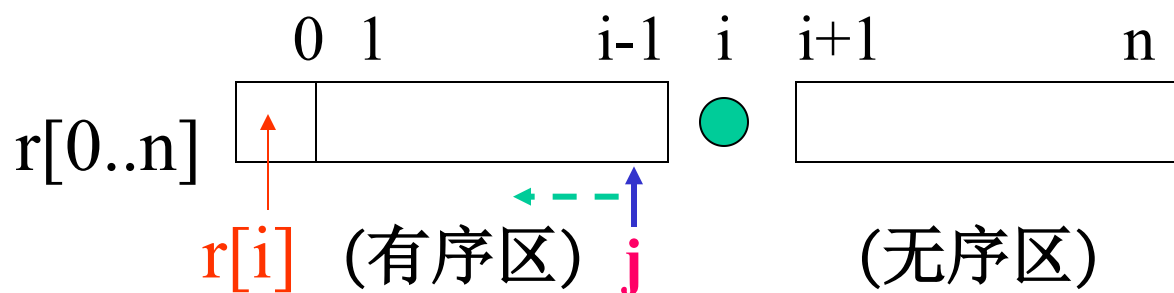
1. 在 $R[1..i-1]$ 中查找 $R[i]$ 的插入位置，
 $R[1..j].key \leq R[i].key < R[j+1..i-1].key$;
2. 将 $R[j+1..i-1]$ 中的所有记录均后移一个位置；
3. 将 $R[i]$ 插入到 $R[j+1]$ 的位置上。

2个技巧:

- ◆ 从后向前找，边找边移动；
- ◆ 把 $R[i]$ 放在 $R[0]$ 作为哨兵，减少比较次数



[算法步骤]



循环 $(n-1)$ 次，初值 $i=2$

1) 若 $r[i] < r[i-1]$ ，则把第 i 个记录取出保存在 $r[0]$ 中， $j=i-1$

2) 若 $r[0] < r[j]$ ，则 $r[j]$ 后移一位， $j=j-1$ ，转2)；

否则 $r[0]$ 放在 $r[j+1]$ 处， $i=i+1$ ，转1)

[算法描述]

```

void InsertSort(SqList& L)
{
    for (int i = 2; i <= L.length; i++)
        if (L.r[i].key < L.r[i - 1].key)
        {
            L.r[0] = L.r[i];
            L.r[i] = L.r[i - 1];//if里比较过了，不在for里比较了
            int j;
            for (j = i - 2; L.r[0].key < L.r[j].key; j--)
                L.r[j + 1] = L.r[j];
            L.r[j + 1] = L.r[0];
        } //if
//InsertSort
}

```

```

void InsertSort2(SqList& L)
{
    for (int i = 2; i <= L.length; i++)
    {
//使用哨兵的顺序查找
        L.r[0] = L.r[i];
        int j;
        for (j = i - 1; L.r[0].key < L.r[j].key; j--)
            L.r[j + 1] = L.r[j]; //对哨兵查找增加的一边查找一边移动
        L.r[j + 1] = L.r[0];
    }//InsertSort
}

```

```

int Seq_Search_2 ( SSTable ST, KeyType key)
{
    ST.elem[0].key=key;
    for (i=ST.length; ST.elem[i].key!=key; i--)
        ;
    return i;
}//Search_Seq2

```



[哨兵/监视哨的作用]

简化边界条件的测试，提高算法时间效率。

[性能分析]

- 最好情况(原始数据按**正序**即非递减序排列)

$$C_{\min} = \sum_{i=2}^n 1 = n - 1 \quad M_{\min} = 0$$

- 最坏情况(原始数据按**逆序**即非递增序排列)

$$C_{\max} = \sum_{i=2}^n i = \frac{(n+2)(n-1)}{2} \quad M_{\max} = \sum_{i=2}^n (i+1) = \frac{(n+4)(n-1)}{2}$$

- 随机情况

$$C_{\text{avg}} = (C_{\min} + C_{\max})/2 \approx n^2/4 \quad M_{\text{avg}} \approx n^2/4$$

- 时间复杂度 $O(n^2)$

辅助空间复杂度 $O(1)$

[改进措施]

- 折半插入排序

算法思想：将循环中每一次在区间 $[1, i-1]$ 上为确定插入位置的顺序查找操作改为折半查找操作。

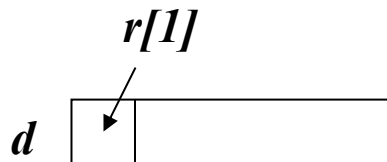
效果：减少关键字间的比较次数。

- 2-路插入排序

算法思想：设置与 r 同样大小的辅助空间 d ，将 $r[1]$ 赋值给 $d[1]$ ，将 d 看作循环向量。

对于 $r[i]$ ($2 \leq i \leq n$)，若 $r[i] \geq d[1]$ ，则插入 $d[1]$ 之后的有序序列中，反之则插入 $d[1]$ 之前的有序序列中。(避免 $r[1]$ 关键字最小/最大)

效果：减少记录的移动次数。



2-路插入排序结果							
(49	<u>49</u>	65	76	97	13	27	38)
					↑ final	↑ first	

• 表插入排序

算法思想：构造静态链表，用改变指针来代替移动记录操作（和链式存储的插入排序方法一样，取未排序序列的第一个key，在已经排序序列里面找插入位置）

```
void
{
    if(Head->next){
        p=Head->next->next;
        Head->next->next = NULL;
    }
    else return;
    while(p)
    {
        pTemp = Head->next; // L.p[0]相当于头结点作用
        pPre = Head;        // 存储插入时，需要pre指针
        //从链表头向后找
        //不能采用顺序存储的从后向前找的哨兵方法
        while (pTemp && (p->data.key >= pTemp->data.key))
        {
            pPre = pTemp;
            pTemp = pTemp->next;
        }
        q=p->next;
        p->next = pPre->next;
        pPre->next=p;
        p=q;
    }
}
```

文。

数组下标	0	1	2	3	4	5
------	---	---	---	---	---	---

初始	0	28	6	59	95	91
	1	0	0	0	0	0

第一趟	0	28	6	59	95	91
	2	0	1	0	0	0

第二趟	0	28	6	59	95	91
	2	3	1	0	0	0

第三趟	0	28	6	59	95	91
	2	3	1	4	0	0

第四趟	0	28	6	59	95	91
	2	3	1	5	0	4

链式存储下的插入排序，对应代码修改

• 表插入排序

算法思想：构造静态链表，用改变指针来代替移动记录操作（和链式存储的插入排序方法一样，取未排序序列的第一个key，在已经排序序列里面找插入位置）

效果：减少记录的移动次数。

```
void InsertSortPoint(Sqlist& L)
```

```
{ PNode p = Head->next;
```

```
Head->next = NULL;
```

```
while(p)
```

```
{
```

```
    pTemp = Head->next; // L.p[0]相当于头结点作用
```

```
    pPre = Head;          存储插入时，需要pre指针
```

```
    //从链表头向后找
```

```
    //不能采用顺序存储的从后向前找的哨兵方法
```

```
    while (pTemp && (p->data.key >= pTemp->data.key))
```

```
    {
```

```
        pPre = pTemp;
```

```
        pTemp = pTemp->next;
```

```
    }
```

```
    q=p->next;
```

```
    p->next = pPre->next;
```

```
    pPre->next=p;
```

```
}
```

```
    p=q;
```

```
}
```

数组下标	0	1	2	3	4	5
------	---	---	---	---	---	---

初始	0	28	6	59	95	91
	1	0	0	0	0	0

第一趟	0	28	6	59	95	91
	2	0	1	0	0	0

第二趟	0	28	6	59	95	91
	2	3	1	0	0	0

第三趟	0	28	6	59	95	91
	2	3	1	4	0	0

第四趟	0	28	6	59	95	91
	2	3	1	5	0	4

链式存储下的插入排序，不是像顺序存储一样把第一个结点当作自然有序了，而是向Head为头的空链表里面插，代码简洁一些。

10.2.2 希尔排序(渐减/缩小增量排序)

对待排记录序列先作“宏观”调整，再作“微观”调整。

[算法思想的出发点]

- 直接插入排序在待排序列的关键字基本有序时，效率较高
- 在待排序的记录个数较少时，效率较高

[算法思想]

先选定一个记录下标的增量 d ，将整个记录序列按增量 d 从第一个记录开始划分为若干组：

第一组为：(1, $1+d$, $1+2*d$, ...),

第二组为：(2, $2+d$, $2+2*d$, ...)

第 d 组为：(d , $2d$, $3d$, ...)

对每组使用直接插入排序的方法进行排序；

然后减小增量 d ，不断重复上述过程，如此下去，直到 $d=1$ (此时整个序列是一组)。

[示例]

<初态>

$d_1=5$

46 82 52 40 67 31 40 73 53 76

31

46

40

82

52

73

40

53

67

76

<第一趟结果>

$d_2=3$

31 40 52 40 67 46 82 73 53 76

31

40

76

82

40

67

73

46

52

53

<第二趟结果>

$d_3=1$

<第三趟结果>

31 40 46 40 67 52 76 73 53 82

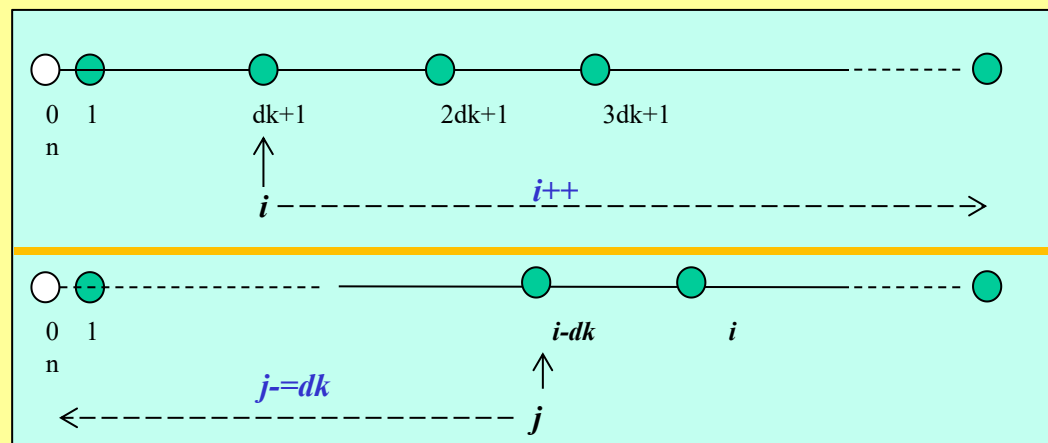
31 40 40 46 52 53 67 73 76 82

不稳定排序

```
void ShellSortN (SqList &L, int dlta[], int t)
//按增量序列dlta[0..t-1]对顺序表L作希尔排序
{
    for (int k = 0; k < t; k++)
    {
        int dk = dlta[k];    //增量序列值，如：.....15,7,3,1
        for( int s = 1; s <= L.length/dk; s++) //对每个序列进行插入排序
        {
            for (int i = dk + s; i <= L.length; i += dk) //单个序列内所有元素
            {
                if (L.r[i].key < L.r[i - dk].key)
                {
                    L.r[0].key = L.r[i].key; //r[i]需插入时，暂存在r[0]
                    int j;
                    for (j = i - dk; j > 0 && L.r[0].key < L.r[j].key; j -= dk)
                    {
                        L.r[j + dk].key = L.r[j].key;
                    }//for j
                    L.r[j + dk].key = L.r[0].key;
                }
            }//for i
        }//for s
    }// for k
}//ShellSortN
```

[算法描述]

```
void ShellSort (SqList &L, int dlta[], int t)
//按增量序列dlta[0..t-1]对顺序表L作希尔排序
{
    for (k = 0; k < t; k++)
    {
        dk = dlta[k];    //增量序列值, 如: .....15,7,3,1
        for (i=dk+1; i <=L.length; i++)
        {
            if (L.r[i].key < L.r[i-dk].key)
            {
                L.r[0] = L.r[i];    //r[i]需插入时, 暂存在r[0]
                for (j=i-dk; j>0 && L.r[0].key<L.r[j].key; j -= dk)
                {
                    L.r[j+dk] = L.r[j];
                    L.r[j+dk] = L.r[0];
                }
            }
        }
    }
} //ShellSort
```



[性能分析]

- 时间复杂度取决于 n 和 $dlta[]$ 序列
- 如何选择最佳 $dlta[]$ 序列，目前尚未解决
 - 最后一个增量值必须为1
 - 避免增量序列中的值（尤其是相邻的值）有公因子
- 实验结果：当 n 较大时，比较和移动次数约在 $n^{1.25}$ 到 $1.6n^{1.25}$ 。
- 就地排序

[直接插入排序代码 和 shell排序一趟dk插入排序对比]

```
void InsertSort(SqList &L)
{
    int i,j;
    for(i=2; i<=L.length; ++i)
    {
        if( L.r[i].key<L.r[i-1].key)
        {
            L.r[0]=L.r[i];
            L.r[i]=L.r[i-1];
            for(j=i-2;
                L.r[0].key<L.r[j].key;
                --j)
                L.r[j+1]=L.r[j];
            L.r[j+1]=L.r[0];
        }
    }
}
```

```
void ShellInsert(SqList &L, int dk)
{
    int i,j;
    for(i=dk+1; i<=L.length; ++i)
    {
        if(L.r[i].key < L.r[i-dk].key)
        {
            L.r[0]=L.r[i];

            for(j=i-dk;
                j>0&&(L.r[0].key<L.r[j].key);
                j=j-dk)
                L.r[j+dk]=L.r[j];
            L.r[j+dk]=L.r[0];
        }
    }
}
```

10.3 交换排序

基本思想:

两两比较，如果发生逆序则交换，直到所有记录都排好序为止。



起泡排序 $O(n^2)$
快速排序 $O(n \log_2 n)$

起泡排序 冒泡排序

将**两个相邻记录**的关键字进行比较，若为**逆序**
则交换两者位置，小者往上浮，大者往下沉。

65 38 27 97 76 03 11 38

38 65 27 97 76 03 11 38

38 27 65 97 76 03 11 38

38 27 65 97 76 03 11 38

38 27 65 76 97 03 11 38

38 27 65 76 03 97 11 38

38 27 65 76 03 11 97 38

38 27 65 76 03 11 38 97

至此，完成**一趟**起
泡；一趟起泡，把
剩余要排序序列中
的**最大值挤到最后**

起泡排序 冒泡排序

将**两个相邻记录**的关键字进行比较，若为**逆序**
则交换两者位置，小者往上浮，大者往下沉。

	65	38	27	97	76	03	11	38	
第一趟起泡	38	27	65	76	03	11	<u>38</u>	97	
第二趟起泡	27	38	65	03	11	<u>38</u>	76	97	稳定排序
第三趟起泡	27	38	03	11	<u>38</u>	65	76	97	
第四趟起泡	27	03	11	38	<u>38</u>	65	76	97	
第五趟起泡	03	11	27	38	<u>38</u>	65	76	97	
第六趟起泡	03	11	27	38	<u>38</u>	65	76	97	

优点：每趟结束时，不仅能挤出一个最大值到最后面位置，还能同时部分理顺其他元素；

一旦此趟没有交换，可提前结束排序

[算法描述]

```
void BubbleSort (SqList &L)
{
    int n=L.length;
    bool sorted=FALSE;
    for(i=0; (i<n-1 && (!sorted)); i++)
    {
        sorted=TRUE;
        for (j=1;j<n-i;j++)
            if (L.r[j].key>L.r[j+1].key)
            {
                L.r[j]↔L.r[j+1];
                sorted=FALSE;
            }//if
    }//for
}// BubbleSort
```

[性能分析]

- 最好情况(原始数据按正序即非递减序排列)

$$C_{\min}=n-1 \quad M_{\min}=0$$

- 最坏情况(原始数据按逆序即非递增序排列)

$$C_{\max} = \sum_{i=1}^{n-1} (n-i) = \frac{1}{2}(n^2 - n) \quad M_{\max} =$$

- 时间复杂度 $O(n^2)$
辅助空间复杂度 $O(1)$

$$3 \sum_{i=1}^{n-1} (n-i) = \frac{3}{2}(n^2 - n)$$

[算法的改进]

- 每趟排序中，记录最后一次发生交换的位置
例：{54, 32, 16, 55, 103, 727, 834, 1006}
- 双向交替扫描，改进数据不对称性：
上→下，最重沉底；下→上，最轻升顶。
例：{12, 18, 42, 44, 45, 67, 94, 10}

10.3.2 快速排序(分划交换排序/分治法)

[分治算法原理]

- 1)分解：将原问题分解为若干子问题
- 2)求解：递归地解各子问题，若子问题的规模足够小，则直接求解
- 3)组合：将各子问题的解组合成原问题的解

[快速排序算法思想]

指定**枢轴/支点/基准记录** $r[p]$ (通常为第一个记录)，通过一趟排序将其放在正确的位置上，它把待排记录分割为独立的两部分，使得

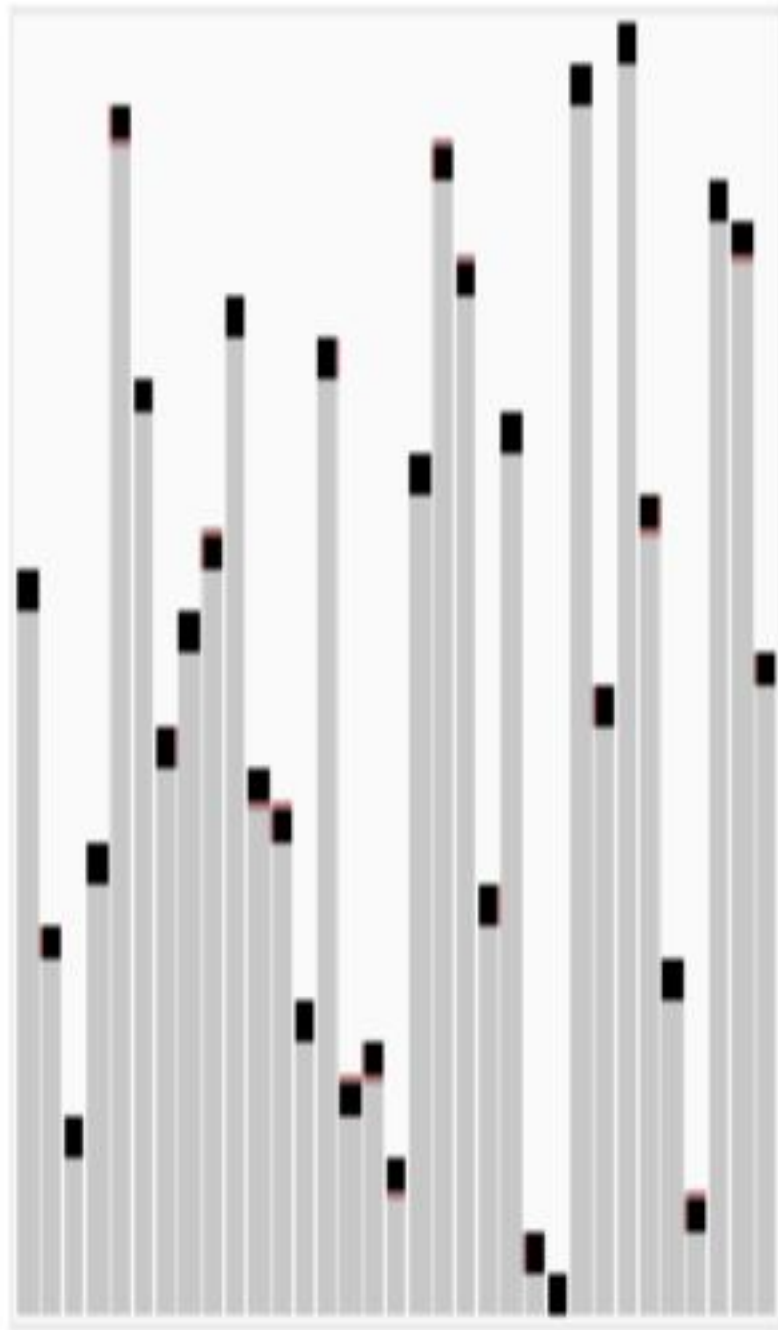
左边记录的关键字 $\leq r[p].key \leq$ 右边记录的关键字

对左右两部分记录序列重复上述过程，依次类推，直到子序列中只剩下一个记录或不含记录为止。(可以用递归方法实现)

快速排序

基本思想：

- 任取一个元素（如第一个）为**枢轴/支点/基准记录**
- 所有比它**小**的元素一律前放，比它**大**的元素一律后放，形成**左右两个子表**；
- **递归的**对各子表重新选择**枢轴**元素并依此规则调整，直到每个子表的元素**只剩一个**



[一趟快速排序示例]



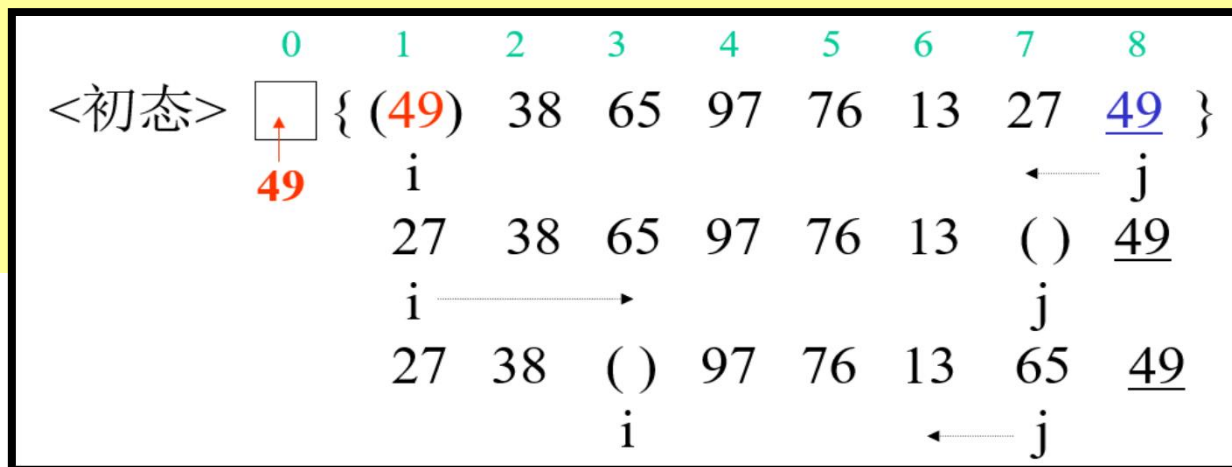
[算法描述]

```
void QuickSort ( SqList &L )  
// 对顺序表 L 进行快速排序  
{  
    QSort ( L, 1, L.length );  
} // QuickSort
```

```
void QSort ( SqList &L, int low, int high )  
//对顺序表 L中的子序列L.r[low..high] 进行快速排序  
{  
    if( low < high )  
    {  
        pivotloc = Partition(L, low, high) ;  
        Qsort (L, low, pivotloc-1) ;  
        Qsort (L, pivotloc+1, high )  
    }  
} // QSort
```

[算法描述] (续)

```
int Partition ( SqList &L, int low, int high )
{
    L.r[0] = L.r[low]; //暂存到0位置, 同时将low位置空出待填
    pivotkey = L.r[low].key;
    while( low < high )
    {
        while( low < high && L.r[high].key >= pivotkey)
            --high; //从后向前找小于pivotkey的数据元素的位置;
        L.r[low] = L.r[high];
        while ( low < high && L.r[low].key <= pivotkey)
            ++low; //从前向后找大于pivotkey的数据元素的位置;
        L.r[high] = L.r[low];
    }
    L.r[low]=L.r[0];
    return low;
} //Partition
```



[性能分析]

- 最坏情况(原始数据正/逆序排列)

$$C_{\max} = (n-1) + (n-2) + \dots + 1 = n(n-1)/2 \quad M_{\max} \leq C_{\max} \quad \mathbf{O(n^2)}$$

- 最好情况：每次划分的结果是基准的左、右两个无序子区间的长度大致相等

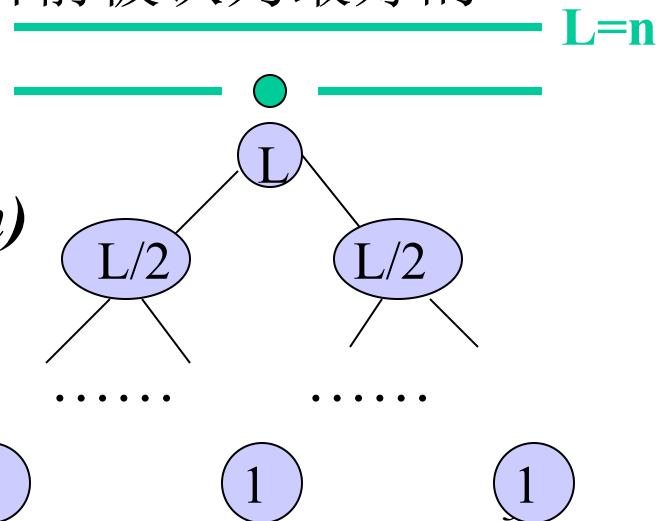
$$C_{\min} \leq O(n \lg n) \quad M_{\min} \leq C_{\min} \quad \mathbf{O(n \log_2 n)}$$

- 平均时间性能 $T_{\text{avg}}(n) = kn \ln(n)$

k: 某个常数; n: 待排序序列中记录个数

就平均时间而言，快速排序是目前被认为最好的一种内部排序方法。

- 辅助空间复杂度 最好情况 $\mathbf{O(\log_2 n)}$
最坏情况 $\mathbf{O(n)}$



[算法的改进]

合理选择枢轴记录可改善性能。例如，

- 三者取中：在1， $n/2$ ， n 三个位置的数中取中间的
- 随机产生：随机产生一个1到 n 的数，取其值

[算法的稳定性]

是非稳定排序

反例 [2, 2, 1]: { 1 2 } **2** { }

思考：在链式存储结构的线性表上如何实现快速排序？

10.4 选择排序

10.4.1 简单选择排序(直接选择排序)

[算法思想]

每次从待排序列中选出关键字最小记录作为有序序列中最后一个记录，直至最后一个记录为止。

[示例] (n=8)

<初态>	49	38	65	97	76	<u>49</u>	13	27
<第1趟>	13	38	65	97	76	<u>49</u>	49	27
<第2趟>	13	27	65	97	76	<u>49</u>	49	38
<第3趟>	13	27	38	97	76	49	49	65
<第4趟>	13	27	38	<u>49</u>	76	97	49	65
<第5趟>	13	27	38	<u>49</u>	49	97	76	65
<第6趟>	13	27	38	<u>49</u>	49	65	76	97
<第7趟>	13	27	38	<u>49</u>	49	65	76	97

不稳定排序

(每趟排序使有序区增加一个记录)

[算法描述]

```
void SelectSort(SqList &L)
{
    for (i=1;i<L.length;i++)
    {
        j=i;
        for(k= i + 1; k<=L.length; k++)
            if(L.r[j].key > L.r[k].key) j= k;
        if(i!=j) L.r[i] ↔ L.r[j];
    }
} //SelectSort
```

[性能分析]

- 总的比较次数与记录排列的初始状态无关

$$C=(n-1)+(n-2)+\dots+2+1=n(n-1)/2$$

- 移动次数 最坏情况: $M_{\max} = 3(n-1)$

$$\text{初始记录正序时: } M_{\min} = 0$$

平均时间复杂度 $O(n^2)$

辅助空间复杂度 $O(1)$

简单选择排
序记录移动
次数少

简单选择排序（直接选择排序）

一趟扫描，选出一个最小值（或最大值）

扫描结束后，将这个最小（或最大值）和占了这个最小值（或最大值）的位置的值进行交换

进行下一次扫描（一共进行 $n-1$ 次扫描）；

最直观，最容易理解的排序

比较次数多，移动次数少

可用于链式存储（头插入或尾插入很方便）

冒泡排序

一趟扫描，将一个最大（或最小）移动到合适的位置

如果此次扫描没有移动，说明已经排序成功，结束

进行下一次扫描（最多进行 $n-1$ 次扫描）；

和相邻位置交换，容易理解

比较次数多，移动次数多

当使用交换节点的值的方式进行交换（不进行节点的摘除和插入）时，也可方便的用于链式存储

10.4.3 堆排序

[特点]

- 较直接选择排序减少重复比较;
- 较树形选择排序减少辅助存储空间(仅需一个)

[(二叉)堆的定义]

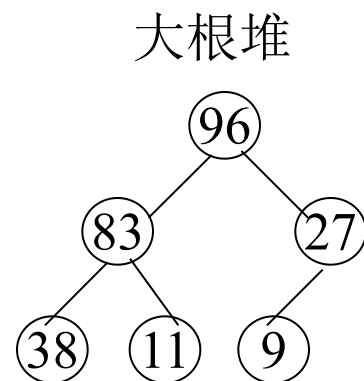
n 个关键字的序列 $(k_1, k_2, \dots, k_n)$ 当且仅当满足下列条件之一:

- (1) $K_i \leq K_{2i}$ 且 $K_i \leq K_{2i+1}$ 小根堆 $(i=1, 2, \dots, \lfloor n/2 \rfloor)$
- (2) $K_i \geq K_{2i}$ 且 $K_i \geq K_{2i+1}$ 大根堆

则称该序列为一个堆。

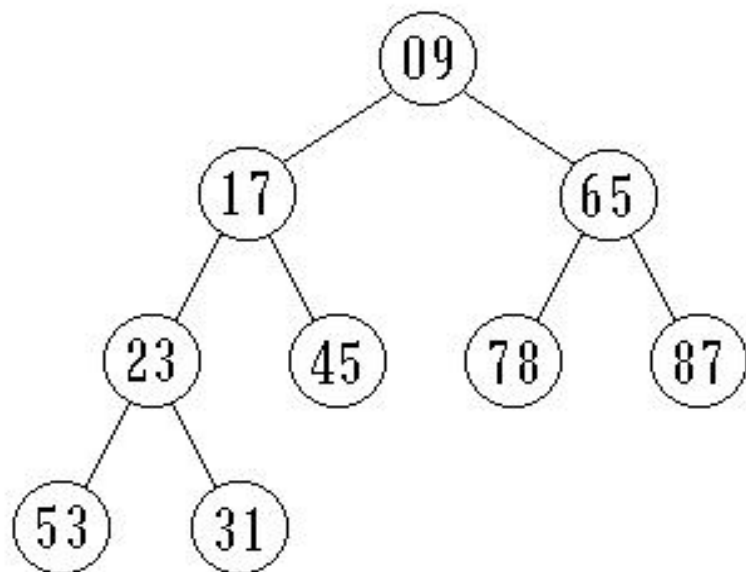
[堆对应于完全二叉树的顺序存储结构]

96	83	27	38	11	9	
1	2	3	4	5	6	



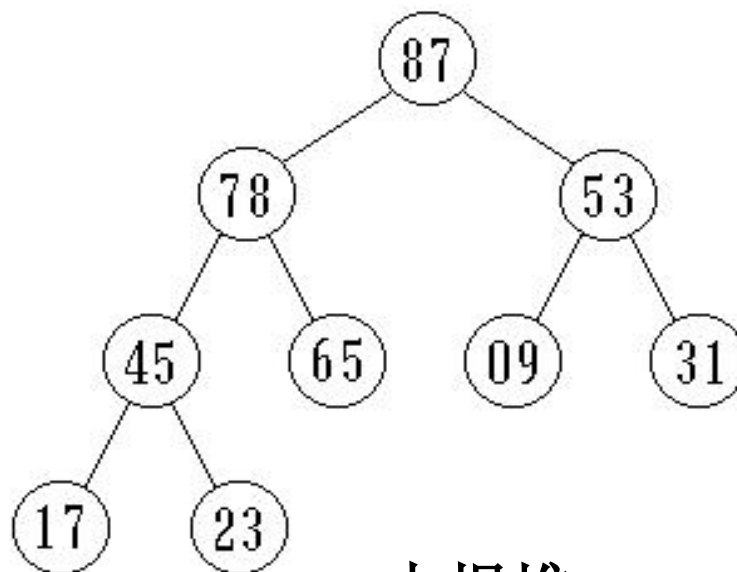
利用树的结构特征来描述堆，所以树只是作为堆的描述工具，堆实际是存放在线形空间中的。

(09,17,65,23,45,78,87,53,31)



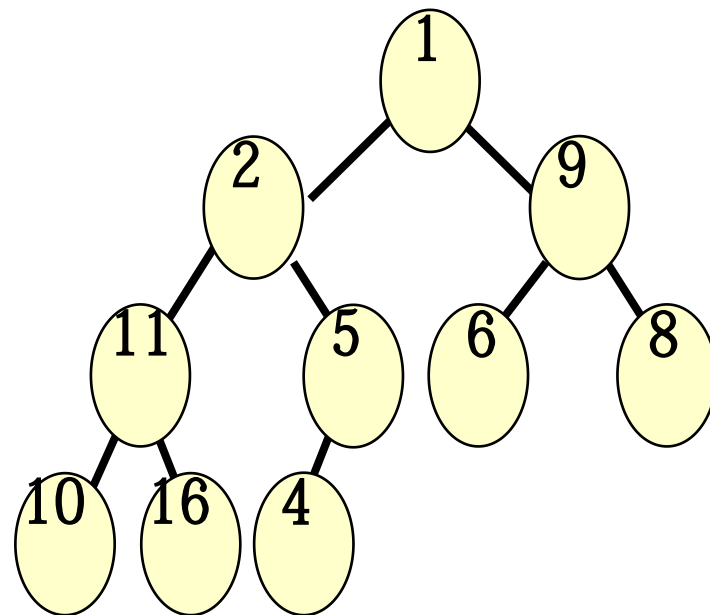
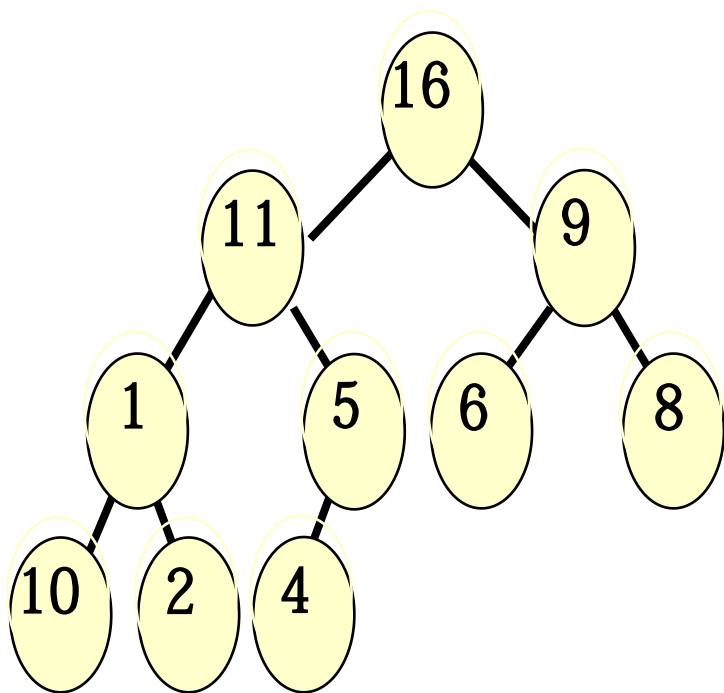
小根堆

(87,78,53,45,65,09,31,17,23)



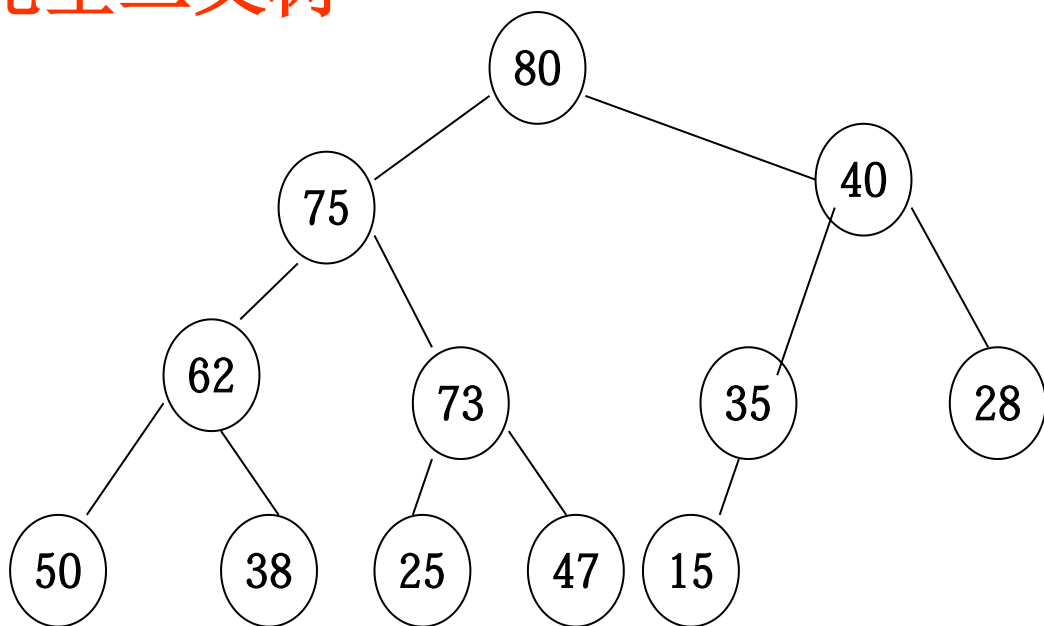
大根堆

堆顶元素（根）为最小值或最大值



判定 (80, 75, 40, 62, 73, 35, 28, 50, 38, 25, 47, 15)
是否为堆

完全二叉树



大根堆

[堆排序基本思想]

第1趟：将初始文件 $R[1..n]$ 建成一个大根堆；

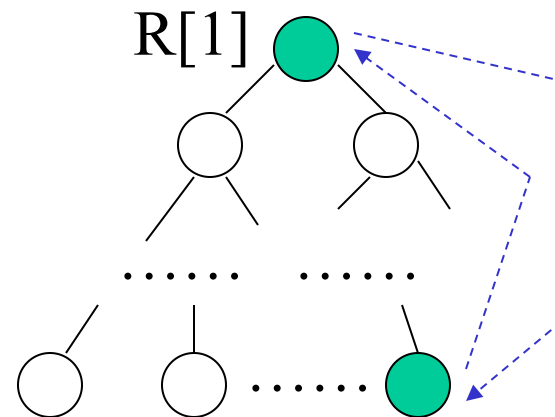
第2趟：将关键字最大的记录(堆顶) $R[1]$ 与无序区的最后一个记录 $R[n]$ 交换；再将新的无序区 $R[1..n-1]$ 调整为堆；

第3趟：将 $R[1]$ 与 $R[n-1]$ 交换；
再将新的无序区 $R[1..n-2]$ 调整为堆；

.....

第 i 趟：将 $R[1]$ 与 $R[n-i+1]$ 交换；
再将新的无序区
 $R[1..n-i]$ 调整为堆；

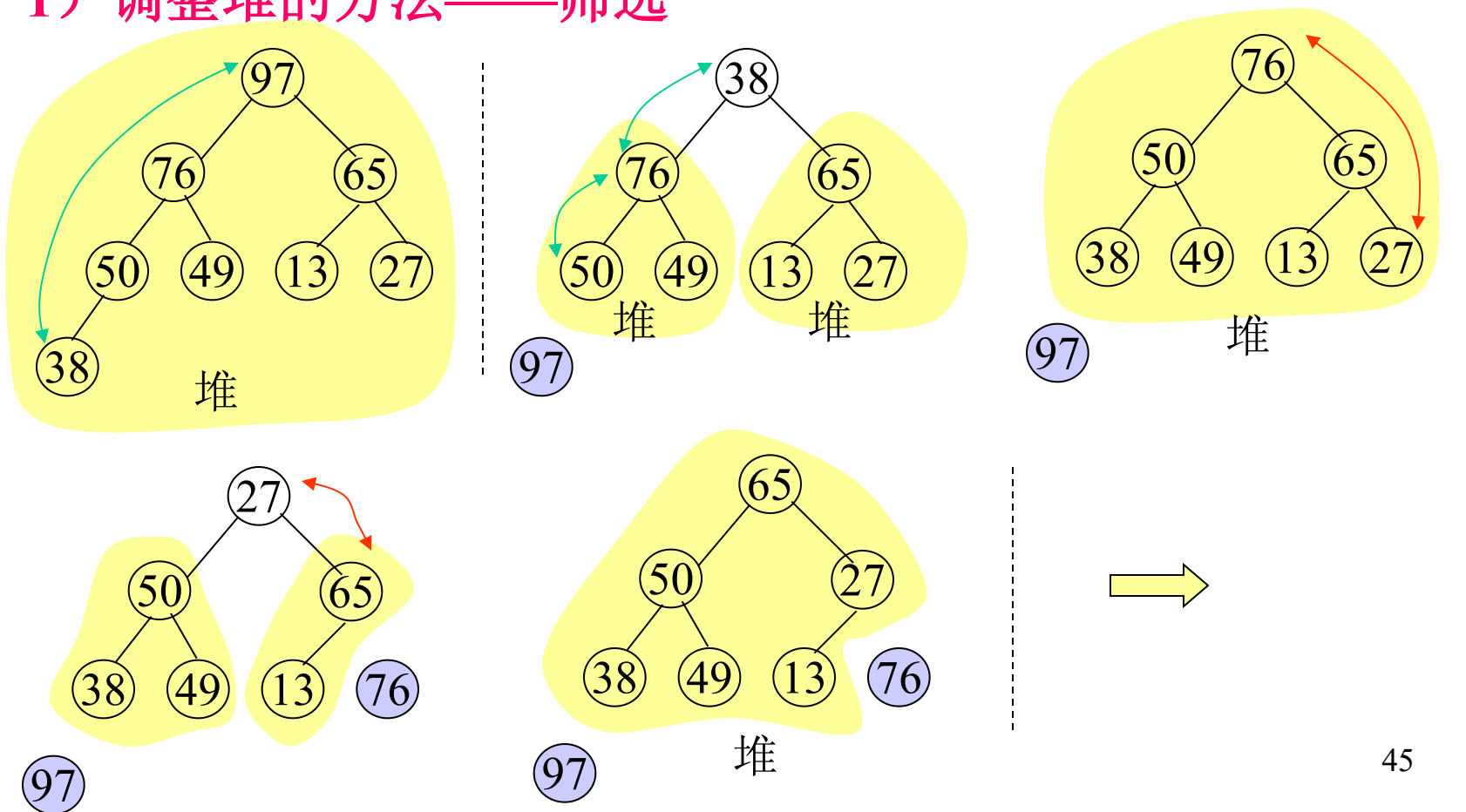
.....直到执行完第 $(n-1)$ 趟为止。



[堆排序需解决的两个问题]

- 如何由一个无序序列建成一个堆？
- 如何在输出堆顶元素之后，调整剩余元素，使之成为一个新的堆？

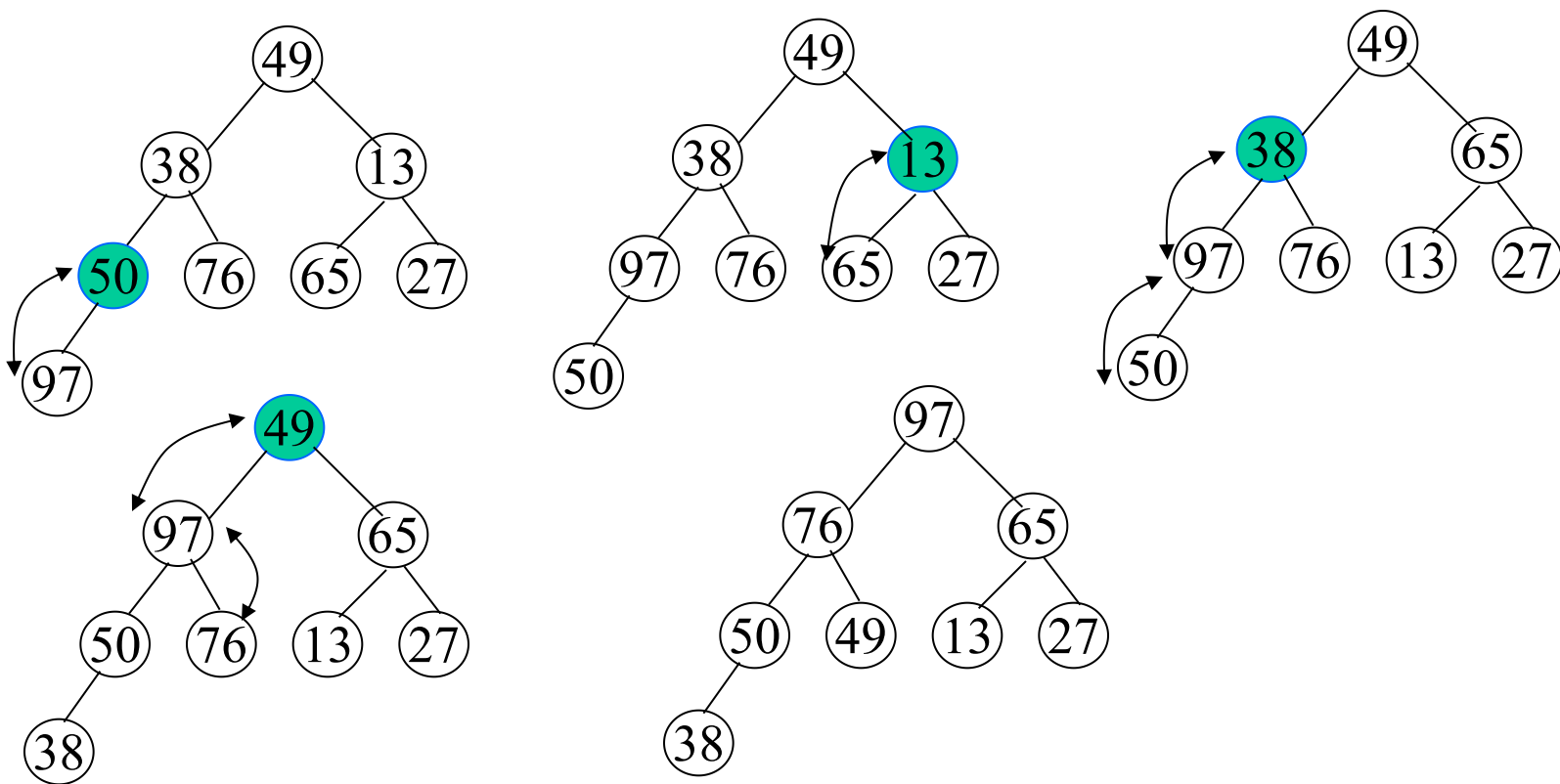
1) 调整堆的方法——筛选



2) 将初始记录序列R[1..n]建成大根堆

依次把以 $i=\lfloor n/2 \rfloor, \lfloor n/2 \rfloor - 1, \lfloor n/2 \rfloor - 2, \dots, 1$ 为根的子树调整为堆，则完成了整个建堆的过程。也称为**第一趟**。

(49, 38, 13, 50, 76, 65, 27, 97)



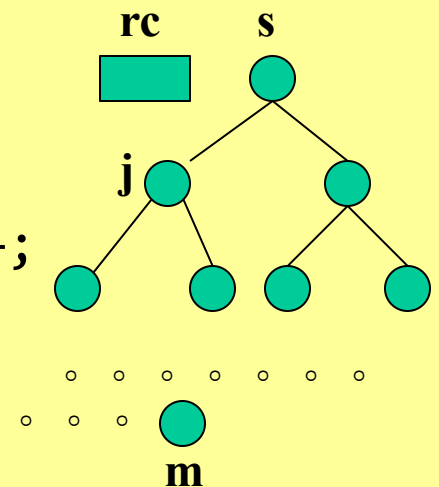
[算法描述]

```
typedef SqList HeapType;
```



```
void HeapSort(HeapType &H)
{
    for (i = H.length / 2; i > 0; i--) //通过不断调整建立堆
        HeapAdjust(H, i, H.length);
    for (i=H.length; i>1; i--) { // 循环进行输出堆顶，调整堆完成排序
        H.r[1]↔H.r[i];
        HeapAdjust(H, 1, i-1);
    }
}
//HeapSort
```

```
void HeapAdjust(HeapType &H, int s, int m)
{
    RedType rc=H.r[s];
    for (j=2*s; j<=m; j*=2) {
        if (j<m && H.r[j].key < H.r[j+1].key) j++;
        if (rc.key > H.r[j].key) break;
        H.r[s] = H.r[j];    s=j;
    }
    H.r[s]=rc;
}
//HeapAdjust
```



[算法分析]

- 最坏情况 $O(n\log_2 n)$
 - 建初始堆时，比较次数 $\leq 4n$
 - 反复调整堆时，比较次数 $< 2n\lfloor \log_2 n \rfloor$
- 实验表明：平均性能接近于最坏性能 $O(n\log_2 n)$
- 辅助空间复杂度： $O(1)$
- 不稳定排序
- 对记录数较大的文件很有效

10.5 归并排序(合并排序)

[归并的概念]

指将两个或两个以上的同序序列归并成一个序列的操作。

10.5.1 两路归并排序

[算法思想]

第1趟：将待排序列 $R[1..n]$ 看作 n 个长度为1的有序子序列，两两归并，得到 $\lceil n/2 \rceil$ 个长度为2的有序子序列(或最后一个子序列长度为1)；

第2趟：将上述 $\lceil n/2 \rceil$ 个有序子序列两两归并；

.....

直到合并成一个序列为止。

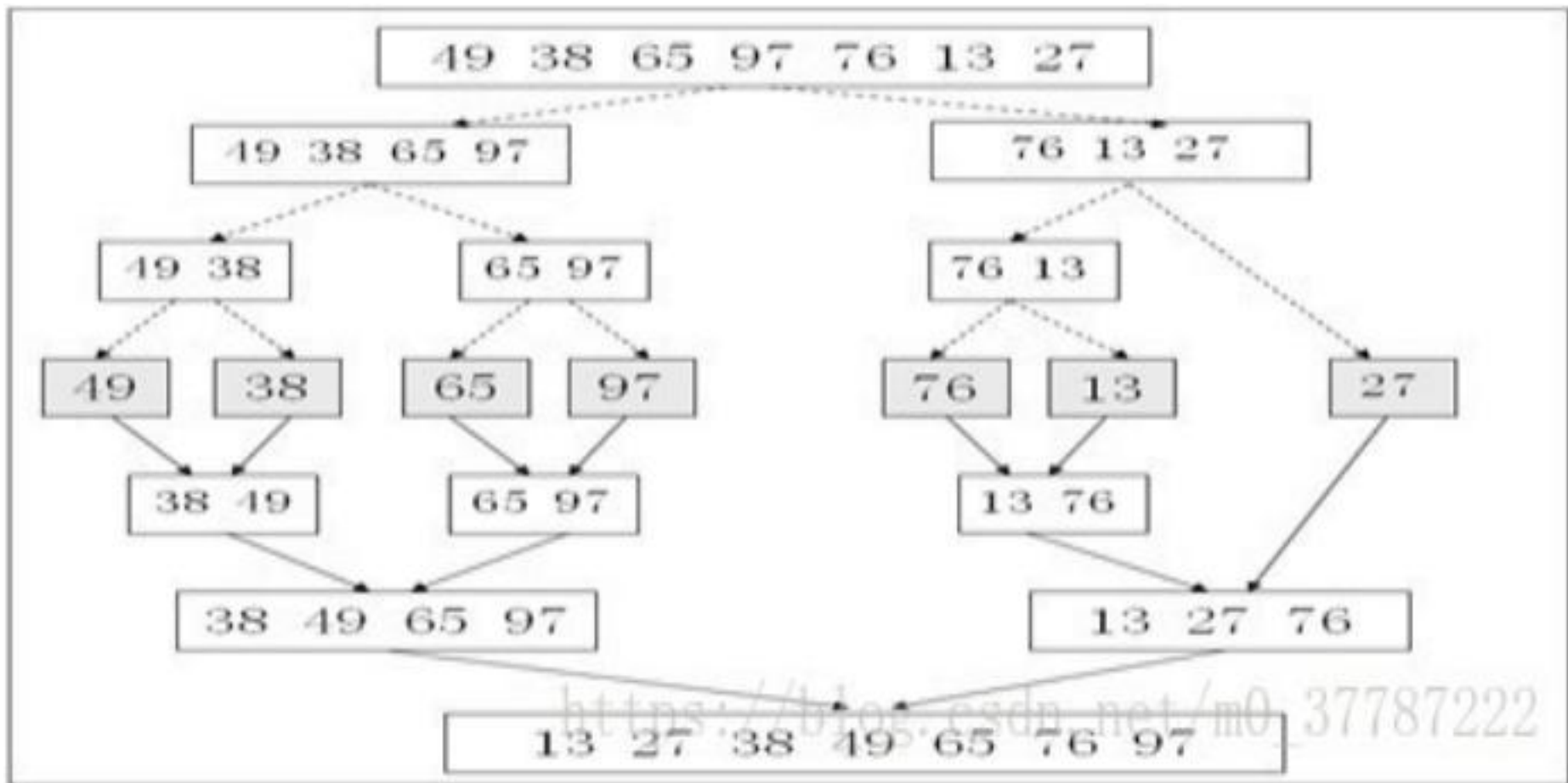
[示例] (n=7)

<初态>	(49)	(38)	(65)	(97)	(76)	(49)	(13)
<第1趟>	(38	49)	(65	97)	(49	76)	(13)
<第2趟>	(38	49	65	97)	(13	49	76)
<第3趟>	(13	38	49	49	65	76	97)

稳定排序

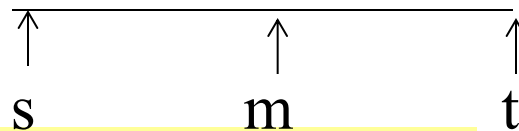
[性能分析]

- 任何情况时间复杂度 $O(n\log_2 n)$
- 空间复杂度 $O(n)$
- 很少用于内部排序



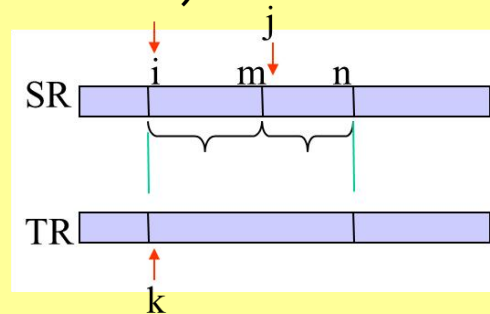
上图中虚线表示递归调用；实线表示递归调用返回。

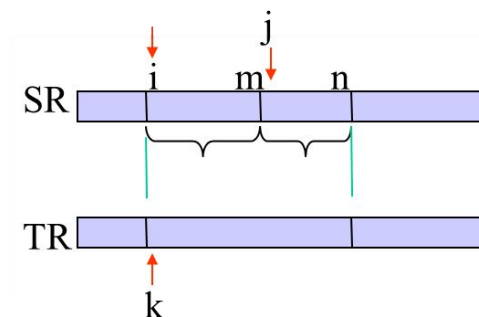
[递归算法描述]



```
void MergeSort(SqList &L) // 将顺序表 L 进行归并排序
{
    MSort(L.r, L.r, 1, L.length ) ;
} // MergeSort
```

```
void MSort( RedType SR[], RedType &TR1[], int s, int t )
// 将 SR[s..t] 归并排序为 TR1[ s..t]
{
    if( s == t )
        TR1[s] = SR[s];
    else
    {
        m = ( s + t ) / 2 ; // 将 SR[s..t] 分为 SR[s..m] 和 SR[m+1..t]
        Msort(SR, TR2, s, m); // 将 SR[s..m] 归并为有序的 TR2[s..m]
        Msort(SR, TR2, m+1, t); // 将 SR[m+1..t] 归并为有序的 TR2[m+1..t]
        Merge(TR2, TR1, s, m, t); // 将 TR2[s..m] 和 TR2[m+1..t] 归并到 TR1[s..t]
    }
} // MSort
```





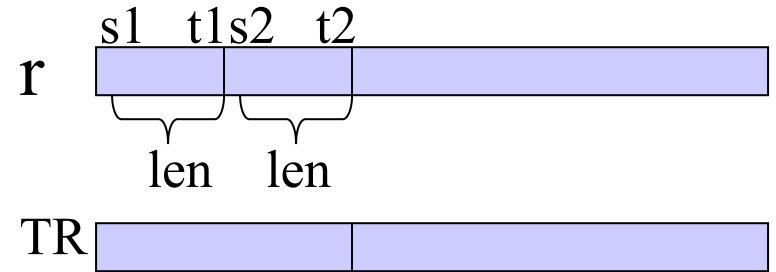
```

Void Merge(RedType SR[], RedType &TR[], int i, int m, int n)
// 将有序的SR[i..m] 和 SR[m+1..n]归并为有序的TR[i..n]
{
    for (j=m+1, k=i; i<=m && j<=n; k++)
    {
        if( SR[i].key <= SR[j].key)
            TR[k] = SR[i++];
        else
            TR[k] = SR[j++];
    }
    if (i<=m) TR[k..n] = SR[i..m];
    if (j<=n) TR[k..n] = SR[j..n];
} //Merge

```

[非递归算法描述]

```
void MergeSort (SqList &L)
{
    int len =1; int n= L.Length;
    while( len < n )
    {
        s1= 1;
        while(s1 + len <= n )
        {
            t1 = s1 +len - 1;    t2 = t1 + len ;
            if (t2 > n)    t2 = n;
            Merge ( L.r, TR, s1, t1, t2);
            s1 = t2+1;
        }
        for(i = 1; i<s1; i++)    L.r [ i ] = TR[ i ];
        len = len*2;
    }
} //MergeSort
```

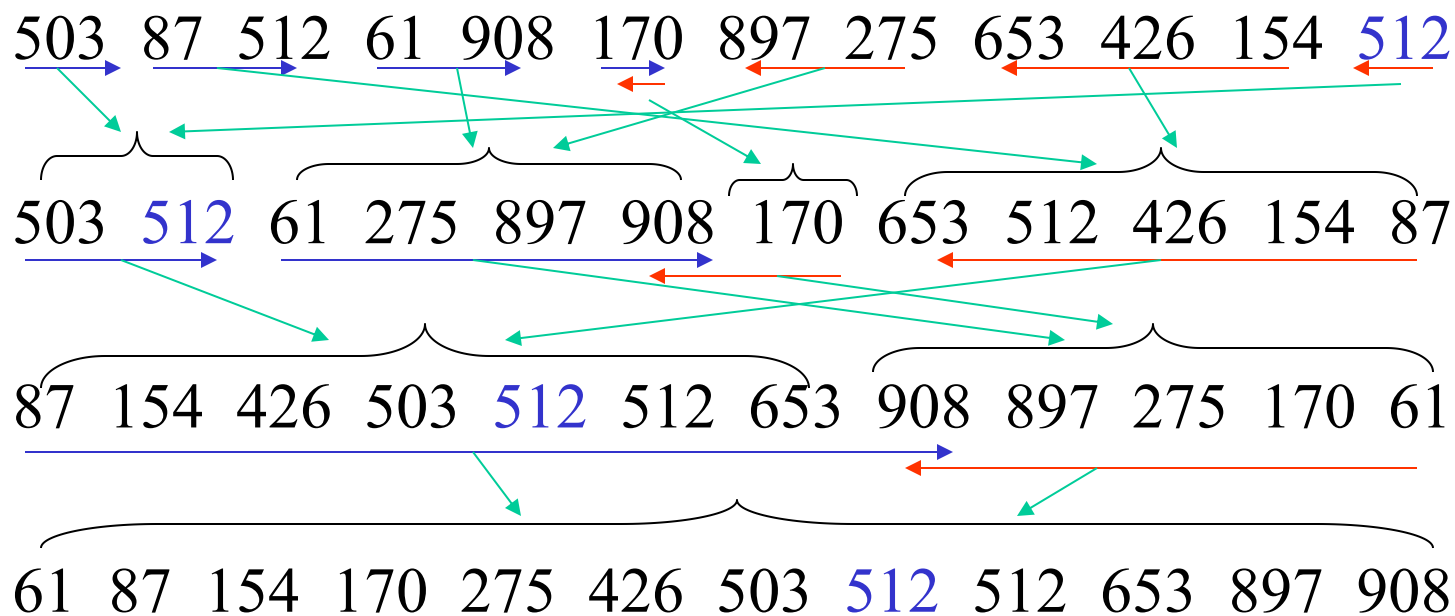


10.5.2 自然两路归并排序

[特点]

以游程(自然的有序段)作为子序列进行归并，可以比直接两路归并更有效。

[示例]



10.6 基数排序(分配排序)

[特点] 通过“分配”和“收集”过程来实现排序，时间复杂度可以突破基于关键字比较一类方法的下界 $O(n\lg n)$ ，达到 $O(n)$ 。

[方法] 借助多关键字排序的思想对单关键字排序

[多关键字排序示例] 扑克牌排序(点数+花色)

双关键字 花色: $\clubsuit < \diamond < \heartsuit < \spadesuit$

点数: $2 < 3 < \dots < Q < K < A$

“花色”地位高于“面值”

可以把所有扑克牌排成以下次序:

$\clubsuit 2, \dots, \clubsuit A, \diamond 2, \dots, \diamond A, \heartsuit 2, \dots, \heartsuit A, \spadesuit 2, \dots, \spadesuit A$ 花色相同的情况下, 比较面值。

最高位优先法 (**MSD法**, Most Significant Digit first)

按花色分成4堆;
对每一堆: 按面值分堆;
从小到大排序;
按花色从小到大排序

最低位优先法 (**LSD法**, Least Significant Digit first)

按点数大小分成13堆;
按点数从小到大收集起来;
再按花色分成四堆;
按花色从小到大收集起来

[MSD与LSD不同特点]

- 按MSD排序, 必须将序列逐层分割成若干子序列, 然后对各子序列分别排序
- 按LSD排序, 不必分成子序列, 对每个关键字都是整个序列参加排序; 并且可不通过关键字比较, 而通过若干次分配与收集实现排序

[基数排序算法思想] 借鉴LSD法

设参加排序的序列为 $K = \{K_1, K_2, \dots, K_n\}$,

其中 K_i 是 d 位 rd 进制的数, rd 称为**基数**;

d 由所有元素中最长的一个元素的位数计量,

$$K_i = K_i^0 K_i^1 \dots K_i^{d-1}$$

高 ← 低

从低位到高位依次对 $K_j (j=d-1, d-2, \dots, 0)$ 根据基数分配, 再按基数递增序收集, 则可得有序序列。

[例]

(1) K

r

(2) K

r

初始状态

053
542
003
063
014
214
154
748
616

按“个位”排序

542
053
003
063
014
214
154
616
748

按“十位”排序

003
014
214
616
542
748
053
154
063

按“百位”排序

003
014
053
063
154
214
542
616
748

<https://blog.csdn.net/developsky024>

[链式基数排序示例]

$$\{ 477 \ 241 \ 467 \ \textcolor{blue}{5} \ 363 \ 81 \ \underline{\textcolor{blue}{5}} \}$$

rd=10
d=3

第1趟：分配 [0] [1] [2] [3] [4] [5] [6] [7] [8] [9]

$$\begin{array}{r} \downarrow 241 \\ 81 \end{array} \quad \begin{array}{r} 363 \\ \downarrow 5 \\ \underline{5} \end{array} \quad \begin{array}{r} 477 \\ \downarrow 467 \end{array}$$

收集 $\{241 \quad 81 \quad 363 \quad 5 \quad \underline{5} \quad 477 \quad 467\}$

第2趟：分配 [0] [1] [2] [3] [4] [5] [6] [7] [8] [9]

$$\begin{array}{r} 5 \\ \downarrow \\ \underline{5} \end{array} \qquad 241 \qquad \begin{array}{r} 363 \ 477 \ 81 \\ \downarrow \\ 467 \end{array}$$

收集 $\{5 \quad \underline{5} \quad 241 \quad 363 \quad 467 \quad 477 \quad 81\}$

第3趟：分配 [0] [1] [2] [3] [4] [5] [6] [7] [8] [9]

$$\begin{array}{r} 5 \\ \downarrow \\ \underline{5} \\ \downarrow \\ 81 \end{array} \qquad \begin{array}{r} 241 \quad 363 \\ \downarrow \\ 477 \end{array}$$

收集 {5 5 81 241 363 467 477 }

稳定排序

$r[0..n]$

key otheritem next

[数据结构]

- 主：静态链表 $r[0..n]$
 - 数据域(key + otheritem)：记录
 - 指针域(next)：下一个记录的序号
- 辅： $f[0..rd-1]$ 各链式队列头指针

	key	otheritem	next
0			1
1			2
2			3
n			0

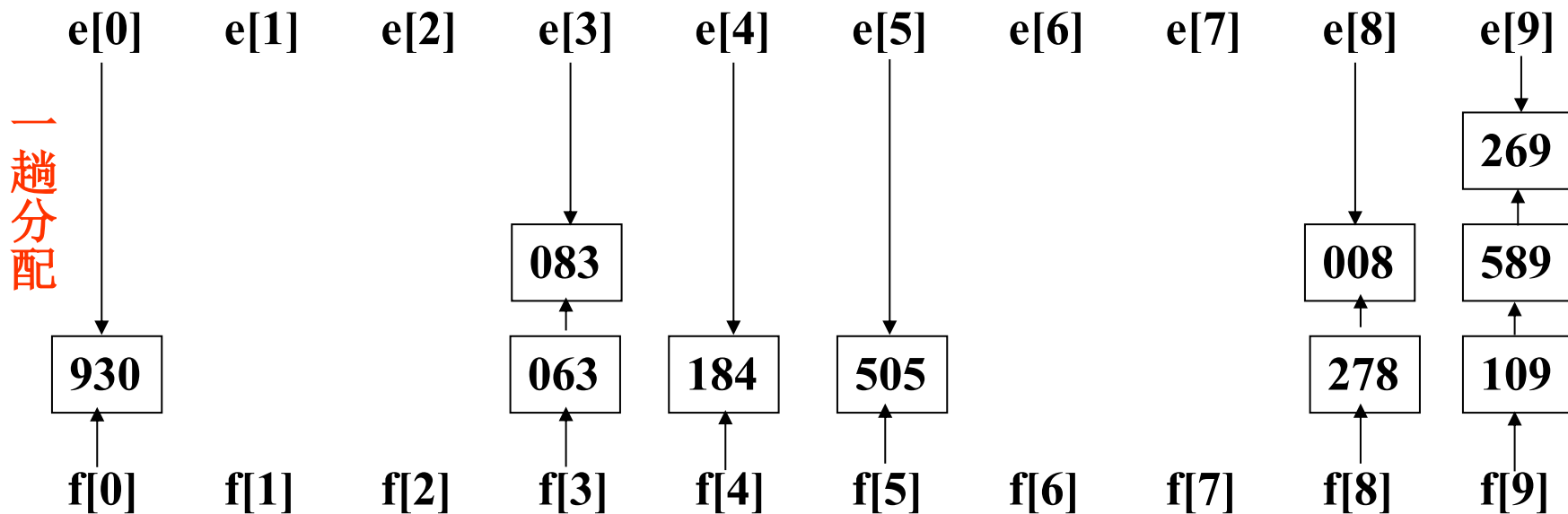
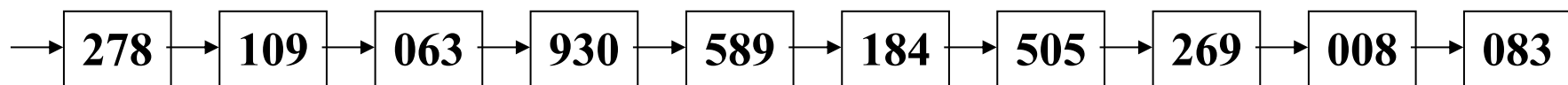
f e

```
#define MAX_NUM_OF_KEY 8 //可容纳的最多子关键字个数
#define RADIX 10 //关键字的基数rd
#define MAX_SPACE 10000 //记录空间可容纳的最多记录数
```

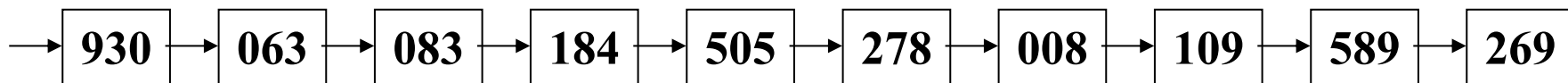
[类]

```
typedef struct {
    KeyType key[MAX_NUM_OF_KEY];
    InfoType otheritem;
    int next;
}SLCell;
typedef struct {
    SLCell r[MAX_SPACE]; //r[0]为头结点
    int keynum ; //实际划分的关键字位数d
    int recnum; //实际记录数
}SLList;
typedef int ArrType[RADIX];
```

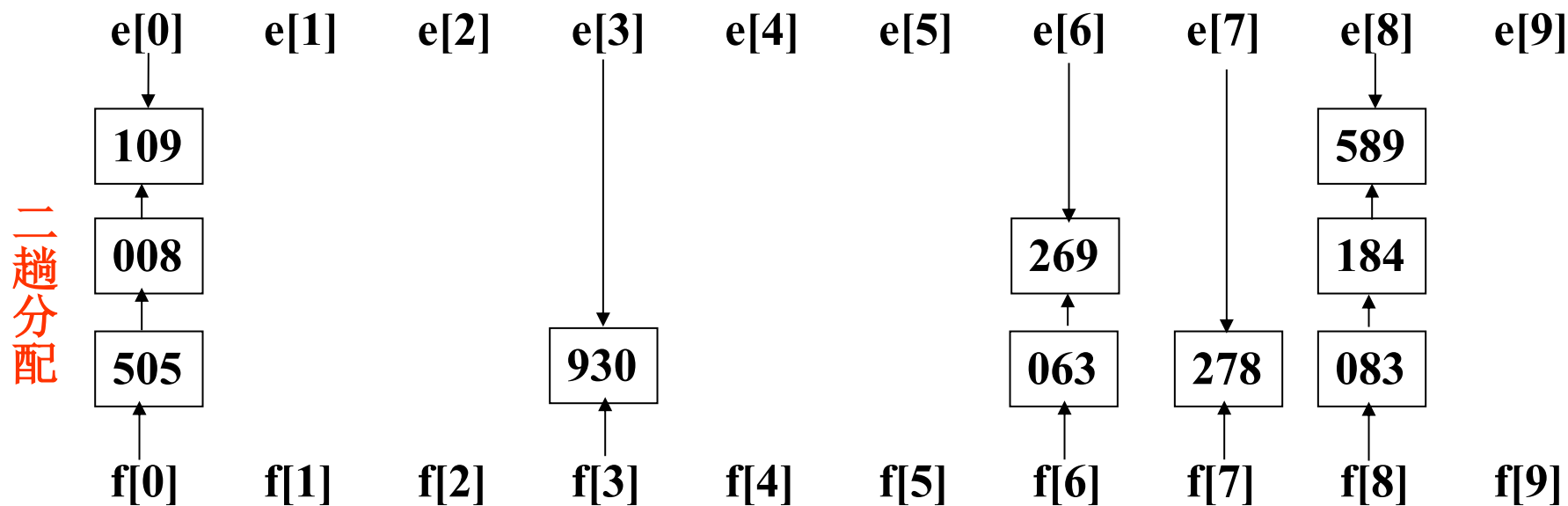
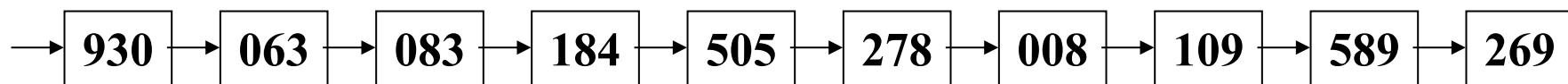
初始状态:



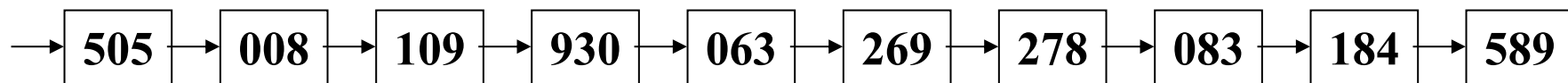
一趟收集:



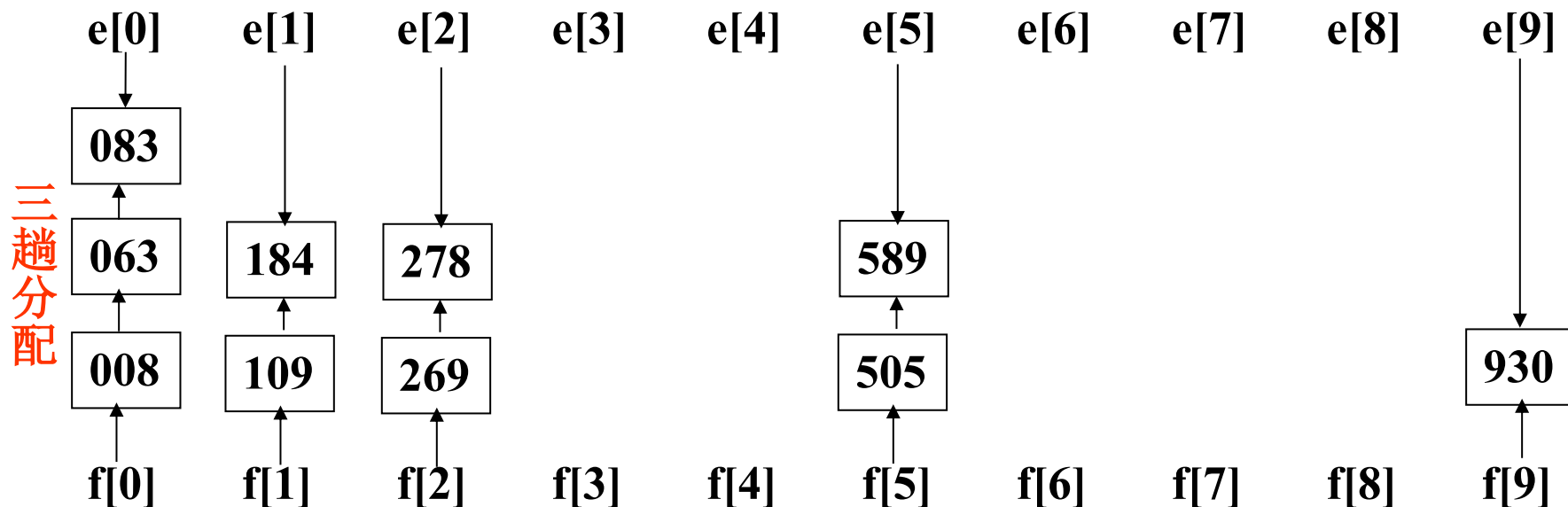
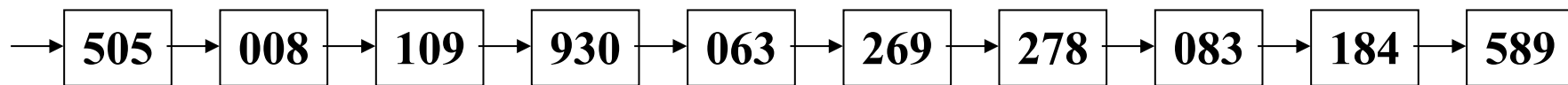
一趟收集



二趟收集:



二趟收集



三趟收集:



[算法描述]

主过程 RadixSort

- 分配过程: Distribute
- 收集过程: Collect

```
void RadixSort (SLList &L)
{
    for(i = 0; i<L.recnum; i++)
        L.r [ i ].next = i+1;
    L.r[L.recnum].next =0;    //将L改造为静态链表
    for (i = 0; i<L.keynum; i++)
    {
        Distribute(L.r, i, f, e);
        Collect(L.r, i, f, e);
    }
}
} //RadixSort
```

```

void Distribute(SLCell &r, int i, ArrType &f, ArrType &e)
{
    for(j=0; j<RADIX; j++)      f[j]=0;
    for(p=r[0].next; p; p=r[p].next) {
        j=ord(r[p].keys[i]);  //求记录中关键字的第i位的队列编号
        if (!f[j]) f[j]=p;
        else r[e[j]].next=p;
    }
}

```

```

void Collect (SLCell &r, int i, ArrType &f, ArrType &e)
{
    for (j=0; !f[j]; j++);  //找第一个非空子队列
    r[0].next=f[j]; t=e[j];
    while (j<RADIX) {
        for (j++; j<RADIX-1 && !f[j]; j++); //找下一个非空子队列
        if ( f[j] ) { r[t].next=f[j]; t=e[j]; } //链接
    }
    r[t].next=0;
} //Collect

```

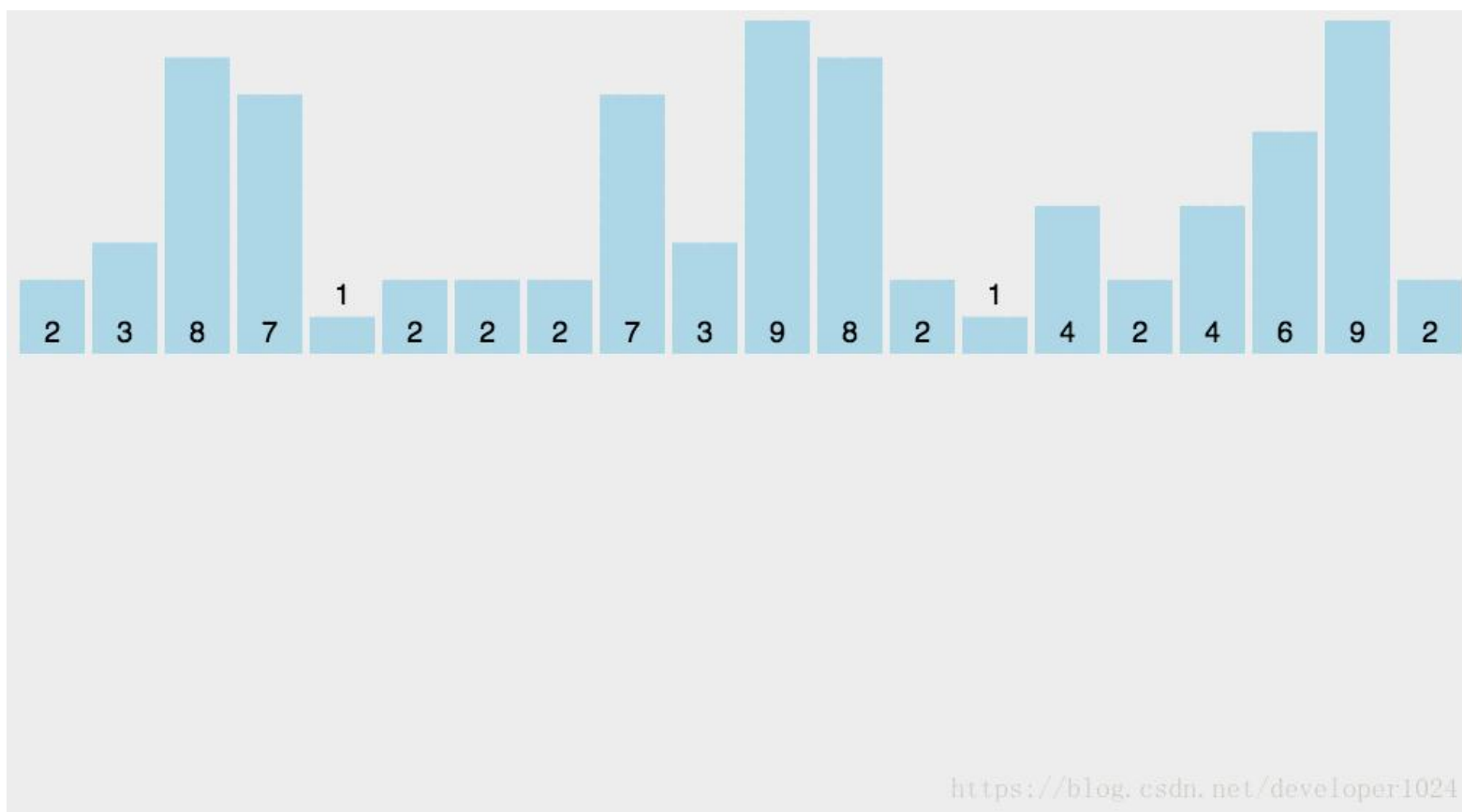
[链式基数排序的性能分析]

设记录数为 n ，关键字位数为 d ，基数为 rd

- 每一趟：分配 $O(n)$ 收集 $O(rd)$
d趟总计： $O(d(n+rd)) \Rightarrow O(n)$ 通常 d, rd 均为常数
- 辅助空间
 - n 个指针域（游标）空间
 - 队头指针数组 $f[0..rd-1]$ 和队尾指针数组 $e[0..rd-1]$辅助空间复杂度： $O(rd+n)$

[计数排序]

- 计数算法只能使用在已知序列中的元素在0-k之间
 - 找出待排序的数组中最大和最小的元素
 - 统计数组中每个值为i的元素出现的次数，存入数组C的第i项
 - 对所有的计数累加（从C中的第一个元素开始，每一项和前一项相加）
 - 反向填充目标数组：将每个元素i放在新数组的第C(i)项，每放一个元素就将C(i)减去1



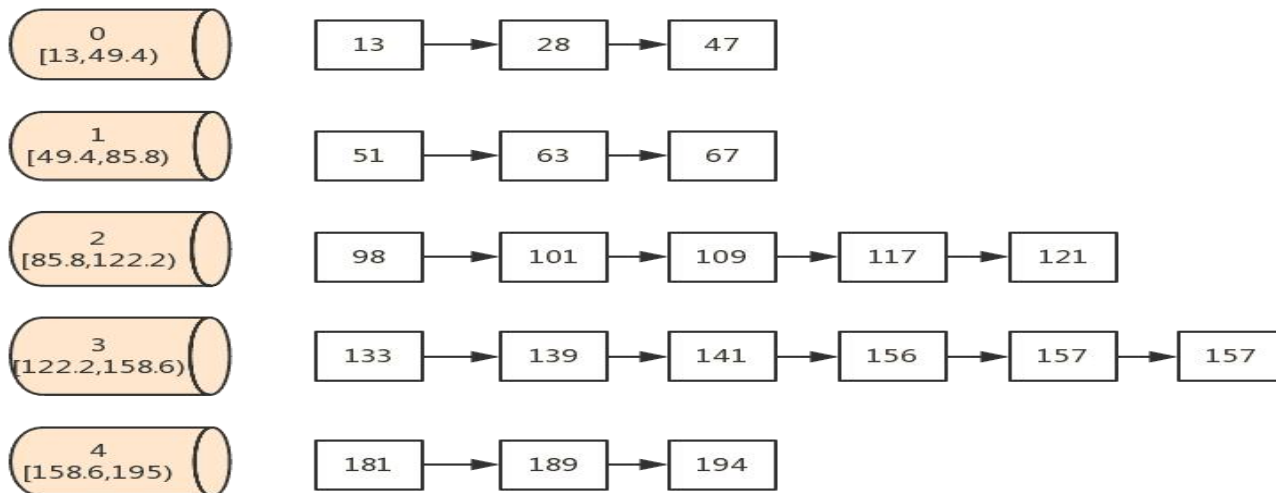
[桶排序]

桶排序（Bucket sort）或所谓的箱排序，是一个排序算法，工作的原理是将数组分到有限数量的桶里。每个桶再个别排序（有可能再使用别的排序算法或是以递归方式继续使用桶排序进行排序），最后依次把各个桶中的记录列出来记得到有序序列。

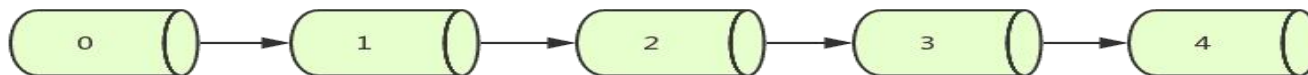
原始数据

63,157,189,51,101,47,141,121,157,156,194,117,98,139,67,133,181,13,28,10
9

分桶数据



合并数据



最终数据

13,,28,47,51,63,67,98,101,109,117,121,133,139,141,156,157,157,181,189,19
4

10.7 各种内部排序方法的比较

排序方法	最好时间	平均时间	最坏时间	辅助空间 (就地排序)	稳定性
直接插入	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	√
希尔		$O(n^{1.3})$		$O(1)$	×
冒泡	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	√
快速	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n^2)$	$O(\log_2 n)$	×
简单选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	×
堆	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	×
归并	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$	√
基数	$O(d(rd+n))$	$O(d(rd+n))$	$O(d(rd+n))$	$O(rd+n)$	√

(数据不是顺次后移时将导致方法不稳定)

- 按平均时间排序方法分为四类
 $O(n^2)$ 、 $O(n\lg n)$ 、 $O(n^{1+\varepsilon})$ 、 $O(n)$
- 快速排序是目前基于比较的内部排序中最好的方法
- 关键字随机分布时，快速排序的平均时间最短，堆排序次之，但后者所需的辅助空间少
- 当 n 较小时如($n < 50$)，可采用直接插入或简单选择排序，前者是稳定排序，但后者通常记录移动次数少于前者
- 当 n 较大时，应采用时间复杂度为 $O(n\lg n)$ 的排序方法(主要为快速排序和堆排序)或者基数排序的方法，但后者对关键字的结构有一定要求
- 当 n 较大时，为避免顺序存储时大量移动记录的时间开销，可考虑用链表作为存储结构（如插入排序、归并排序、基数排序）

- 快速排序和堆排序难于在链表上实现，可以采用地址排序的方法，之后再按辅助表的次序重排各记录
- 文件初态基本按正序排列时，应选用直接插入、冒泡或随机的快速排序
- 选择排序方法应综合考虑各种因素

讨论：假设有 n 个值不同的元素存于顺序结构中，要求不经排序选出前 k ($k \ll n$) 个最小元素，问哪些方法可用，哪些方法比较次数最少？这 k 个元素也要有序如何？

- 选择排序或冒泡排序： k 趟（数据比较次数约为 kn 次）；
- 快速排序：每次仅对第一个子序列划分，直至子序列长度小于等于 k ；长度不足 k ，则再对其后的子序列划分出补足的长度即可（平均对数据比较 $2n$ 次）
- 堆排序：先建小根堆， $k-1$ 次堆调整（数据比较次数约为 $4n + (k-1)\log_2 n$ ）

本章学习要点

1. 了解排序的定义和各种排序方法的特点。熟悉各种方法的排序过程及其依据的原则。基于“关键字间的比较”进行排序的方法可以按排序过程所依据的不同原则分为插入排序、交换排序、选择排序、归并排序和计数排序等五类。
2. 掌握各种排序方法的时间复杂度的分析方法。能从“关键字间的比较次数”分析排序算法的平均情况和最坏情况的时间性能。